

第29回 自分で機能を追加する

今回のゴール

- 追加したい機能を自分で決め、1 文の仕様にできる。
- 仕様から、変更するファイル・メソッドを見つけられる。
- 小さく実装し、実行して確かめ、うまくいかなければ直せる。
- 自分が変えたところと理由を、ほかの人に説明できる。

今回、新しい Java の文法は増やさない。これまでに学んだことだけで、機能を追加する。

前回とのつながり

第28回では、「仕様 → 変更箇所 → 実装 → 実行・確認」の順で、いやしスライムと戦士のよろいを追加した。今回は、何を追加するかを **自分で決める**。

実習時のフォルダ構成

```
ワークスペース
├─ java29_sample
│   └─ src
│       └─ battle
│           └─ (第28回の解答例 + 「にげる」の機能追加の例。9 ファイル)
└─ java29_exercise
    └─ src
        └─ battle
            └─ (第28回の解答例と同じ 9 ファイル。ここに自分の機能を追加する)
```

- 配布ファイル: プログラム/第29回/sample/battle/、プログラム/第29回/exercise/battle/
- 解答例(機能追加の例): プログラム/第29回/answer/battle/
- exercise の Main.java の先頭に、追加した機能をメモする欄がある。

今日の時間の目安

時間	内容
15 分	機能追加の例を読む・進め方の確認
10 分	自分の機能を決めて、仕様シートを書く
45 分	実装・実行・修正(くり返し)
15 分	となりの人に説明する・振り返り

講義内容

1. 機能追加の例:「にげる」を確実にしない

仕様:「にげる」を選んでも、にげられるのは 2 回に 1 回くらいにする。にげられなかったときは、敵の攻撃を受ける。

変更箇所:第27回の「どこを変えると何がかわるか」の表で、「コマンドの種類」は `Main` の `switch`。→ `Main` の `case 3` だけを変える。

実装: プログラム/第29回/sample/battle/Main.java

```
case 3:
    // 【機能追加の例】にげられるのは 2 回に 1 回くらい（乱数で決める）
    if (random.nextInt(2) == 0) {
        System.out.println("パーティーはにげだした！");
        escaped = true;
    } else {
        System.out.println("にげられなかった！");
    }
    break;
```

実行・確認: `3` を何回か選び、「にげだした」と「にげられなかった」の両方が出ることを確かめる。にげられなかったときに、そのまま敵の攻撃に進むことも確かめる(`escaped` が `false` のままなので、`break` しない)。

```
コマンドを選ぶ > 3
にげられなかった！
スライムの攻撃！ 戦士に5のダメージ
...
```

- 変更は `if` と `Random` だけ。どちらも、もう使ったことのある文法。
- 「どこを変えるか」を先に決めたので、ほかのファイルを開く必要がなかった。

2. 機能カード

次の中から選ぶか、自分で考える。最初は **Lv1** か **Lv2** から 始め、できたら次に進む。

レベル	機能の例	主な変更箇所	使う文法
Lv1 値・条件を変える	勇者の薬草を 3 個にする	<code>Main</code> の <code>new Hero(...)</code>	引数
Lv1	僧侶が最初から仲間にいる	<code>Main</code> の <code>party.add</code> と <code>if</code> (<code>turn == 2</code>)	<code>ArrayList</code>
Lv1	敵を追加する・強さを変える	<code>Main</code> の <code>enemies.add</code>	コンストラクタ
Lv1	会心の一撃の間隔を変える	<code>Hero</code> の <code>attack</code>	<code>%</code> 、 <code>if</code>
Lv2 処理を追加する	勝ったとき、生き残った仲間を表示する	<code>Main</code> の結果の表示	拡張 <code>for</code>

レベル	機能の例	主な変更箇所	使う文法
Lv2	「ためる」を 3 回までにする	Main の case 4 と回数の変数	if、変数
Lv2	魔法使いの魔法を、たまに大成功(3 倍)にする	Wizard の attack	Random、if
Lv2	敵をたおすと、全員が HP を 5 回復する	Main の case 1	拡張 for、heal
Lv3 クラスを追加する	新しい敵:ドラゴン(炎をはく)	新しいクラス、Main の enemies.add	継承、オーバーライド
Lv3	新しい職業:騎士(仲間の代わりに攻撃を受ける など)	新しいクラス、Main の party.add	継承、オーバーライド
Lv3	魔法使いも回復できるようにする	Wizard に implements Healer	インタフェース

- 解答例(answer)には、薬草 3 個(Lv1)、生き残った仲間の表示(Lv2)、ドラゴン(Lv3)の例が入っている。自分で書いた後に見比べる。
- ドラゴンを入れると、とても強くなる。敵の強さを調整するのも立派な機能追加。

3. うまいかないときは

- 機能を小さく分ける。「新しい職業を作る」なら、①クラスを作って名前だけ表示 → ②攻撃を変える → ③特別な能力を足す、のように 1 つずつ実行して確かめる。
- エラーが出たら、第26回の手順(いちばん上のエラーから、行番号 → メッセージ → 直前に変えたところ)。
- 動くけれど結果が違うときは、予想した結果をノートに書いてから、どこで違ってしまうのか System.out.println で値を表示して調べる。
- どうしても動かなくなったら、配布ファイルに戻してやり直してよい。

演習1 例を実行する

java29_sample を作って実行し、「にげる」を何回か試す。

演習2 自分で機能を追加する

java29_exercise を作り、battle パッケージに プログラム/第29回/exercise/battle/ の 9 ファイルを入れる。

(配布した状態で、第28回の解答例と同じように動く)

演習2-1 仕様シートを書く

コードを書く前に、次をノートに書く。

項目	書くこと
機能(1 文)	例:「勝ったとき、生き残った仲間の名前を表示する」
レベル	Lv1 / Lv2 / Lv3
変更するファイル・メソッド	例: <code>Main</code> の <code>main</code> の、結果を表示するところ
使う文法	例: 拡張 <code>for</code> 、 <code>getName()</code>
実行してどうなれば完成か	例: 勝ったときに <code>生き残った仲間: 勇者 戦士</code> と表示される

演習2-2 実装・実行・修正

- 仕様シートの「変更するファイル・メソッド」だけを変える。
- 1 か所変えるたびに、保存して実行する。
- `Main.java` の先頭のメモ欄に、機能と変更したところを書く。
- 1 つできたら、次の機能に進む。

演習2-3 説明する

となりの人に、次の順で説明する(1 人 3 分)。

確認	内容
読む	変更する前のコードの、どこを読んだか
予想	変更後に、どうなると予想したか
変更	どのファイルの、どこを変えたか
実行	実行して、どうなったか
修正	エラーや予想と違うことがあったか。どう直したか

聞いた人は、「ほかにどこを変えると、もっとよくなるか」を 1 つ提案する。

発展(任意)

- Lv3 の機能に挑戦する。
- 自分の追加した機能に、わざとまちがいを 1 つ入れて、となりの人に見つけてもらう(第26回の発展と同じ)。

確認問題

問1 次の機能を追加するとき、主に変更するのはどのファイルか。

- 戦士の攻撃を、2 回攻撃にする。
- HP が 0 未満にならないルールを変える。

3. 新しいコマンド「ぼうぎょ」を追加する。

問2 `random.nextInt(2) == 0` が `true` になる確率は、だいたいどれくらいか。

問3 「にげる」の例で、にげられなかったときに `escaped = true;` と書いてしまうと、どうなるか。

■ 答え

- 問1: 1. `Warrior.java` (`attack`) 2. `Character.java` (`damage`) 3. `Main.java` (`commands` と `switch`)
- 問2: 2 回に 1 回くらい (`nextInt(2)` は 0 か 1 を返す)
- 問3: 「にげられなかった！」と表示されるのに、バトルが終わってしまう(論理エラー)

振り返り

- 自分の機能を、1 文の仕様にできたか。
- 変更するファイル・メソッドを、コードを書く前に決められたか。
- 次に追加してみたい機能は何か。

次回へのつながり

今回は、後期のまとめ。第16回から学んだ Java を、短いコードで確認し、最後にもう一度このバトルを読んで変更する。